



UNIVERSIDAD
TÉCNICA DE MANABÍ
Fundada en 1952

DESARROLLO DE SISTEMAS INFORMÁTICOS



UNIDAD 1 - Clase #2



UNIDAD 1

Desarrollo de software Introducción y Paradigmas de diseño

1. Herramientas para la construcción de proyectos de software
2. Herramientas para el control de versiones de software

Tema 2:

1. Diseño Estructurado



UNIDAD 1

Resultados de aprendizaje

- Explicar los conceptos y definiciones a considerar en el desarrollo de un software así como también los aspectos del diseño
- Describir y explicar el uso de herramientas de control de versión de un proyecto de software



Herramientas para la Construcción de Proyectos de Software

IDE - Definición y características

Un entorno de desarrollo integrado (IDE) es un sistema de software que combina herramientas del desarrollador en una sola interfaz gráfica.



Editor de código fuente

Resaltado de sintaxis, autocompletado y detección de errores en tiempo real



Depurador

Localiza errores en el código original de forma gráfica



Automatización de compilación

Convierte código fuente a binario y ejecuta pruebas automatizadas



Integración

Conecta con herramientas externas y sistemas de control de versiones

BRIEF ABOUT IDE (INTEGRATED DEVELOPMENT ENVIRONMENTS)



IDE - Importancia y tipos conocidos

★ ¿Por qué usar un IDE?

Rapidez

Configuración automática de herramientas

Autocompletado

Generación inteligente de código

Detección de errores

Identificación en tiempo real

Integración

Todas las herramientas en un solo lugar

Tipos de IDE

<> Lenguajes compatibles

IDEs especializados (IntelliJ) vs. multiplataforma (Eclipse)

💻 Sistemas operativos

Compatibilidad con Windows, macOS, Linux y plataformas móviles

⚙️ Características de automatización

Refactorización, búsqueda de código, herramientas CI/CD

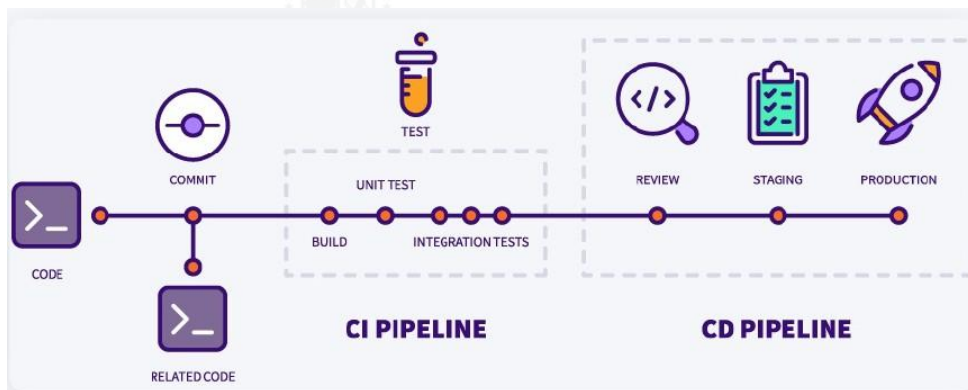
⚙️ Impacto en rendimiento

Footprint en memoria y consumo de recursos del sistema

🧩 Complementos y extensiones

Personalización de flujos de trabajo según necesidades

CI/CD - Concepto y herramientas



¿Qué es CI/CD?

Integración Continua y Entrega Continua: **automatización** del proceso de desarrollo, prueba y despliegue de software.

Velocidad



Entrega más rápida de funcionalidades

Calidad



Detección temprana de errores

Iteración



Ciclos de desarrollo más cortos

Colaboración



Mejor integración entre equipos

"Quality at Speed"

La nueva norma en desarrollo de software que equilibra **rapidez** y **calidad** mediante la automatización del ciclo de vida.



Herramientas más utilizadas



Jenkins



GitLab CI



GitHub Actions



CircleCI



Travis CI



Azure DevOps

Herramientas de modelado UML

¿Qué es UML?

Lenguaje de Modelado Unificado: **abstrae y visualiza** sistemas de programación orientada a objetos, **facilitando la comunicación técnica y no técnica.**



Caso A: Bocetos simples

Herramientas gratuitas para prototipos rápidos. Formas gráficas con significado semántico básico.



Caso B: Sistemas complejos

Simplificación de sistemas complejos para comprensión multidisciplinaria. Facilita trabajo en equipo.



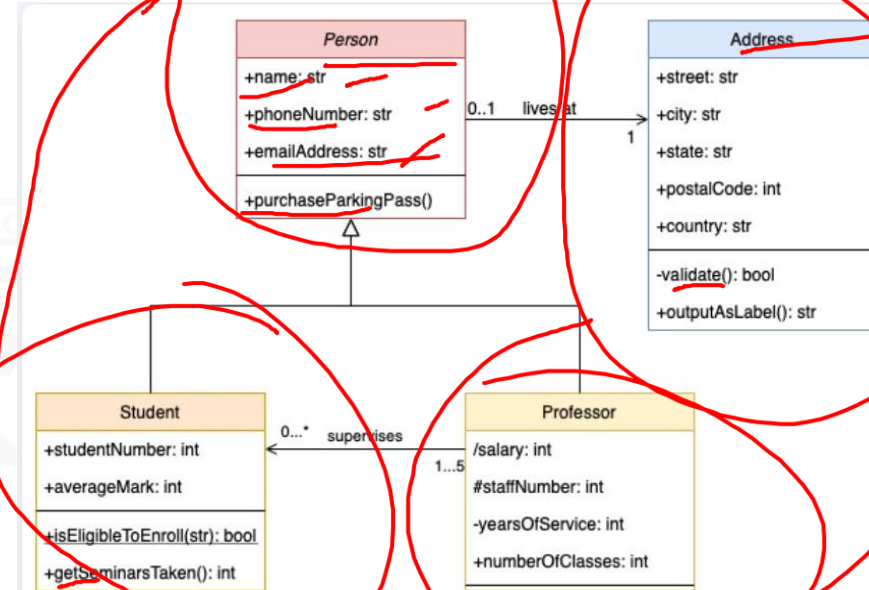
Caso C: Código desde modelo

Conversión de diagramas a lenguajes de programación (Java, C++, C#, XML) y viceversa.



Caso D: Integración

Visualización de interfaces y puntos de conexión para integrar nuevos componentes en sistemas existentes.



Ventajas clave



Visualización clara ✓



Comunicación efectiva ✓



Abstracción de complejidad ✓



Generación de código ✓

Draw!g



UML Tutorial: Diagrama de Clases desde cero



Herramientas para gestión de proyectos de software



★ Beneficios clave

- Visibilidad del progreso
- Colaboración efectiva
- Optimización de recursos
- Automatización de tareas

📋 ¿Qué son?

Programas informáticos que permiten a los gestores **planificar, ejecutar y gestionar** proyectos en una ubicación virtual centralizada.



Planificación

Definición de objetivos y alcance del proyecto



Asignación de recursos

Distribución de personal y equipos



Gestión de calidad

Estándares y control de calidad



Informes

Elaboración y publicación de reportes



Programación

Establecimiento de plazos y cronogramas



Presupuesto

Control de costos y gestión financiera



Documentación

Almacenamiento y compartición de archivos



Análisis

Seguimiento y pronóstico de tendencias



Herramientas para el Control de Desarrollo de software

Definición e importancia del VCS

¿Qué es un VCS?

Sistema que permite **guardar un registro** de las modificaciones realizadas sobre archivos a lo largo del tiempo, facilitando la recuperación de versiones específicas.



Trabajo colaborativo

Permite que múltiples desarrolladores trabajen simultáneamente **sin interferir entre sí**



Historial completo

Mantiene un registro detallado de quién, cuándo y por qué se realizaron cambios



Recuperación de errores

Facilita volver a versiones estables cuando algo sale mal

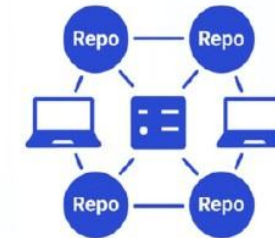
Types of Version Control Systems



Local VCS



Centralized VCS (CVCS)



Distributed VCS (DVCS)

Tipos de VCS



Local



Centralizado



Distribuido



Basado en la nube

Características fundamentales del VCS

Los sistemas de control de versiones ofrecen **funcionalidades esenciales** para gestionar eficientemente el código fuente



Resolución de conflictos

Monitoriza y ayuda a resolver conflictos cuando varios desarrolladores modifican simultáneamente el mismo archivo

Colaboración

Fusión

Detección



Revertir cambios

Permite deshacer rápidamente modificaciones y volver a versiones estables conocidas del código fuente

Historial

Recuperación

Seguridad



Copia de seguridad

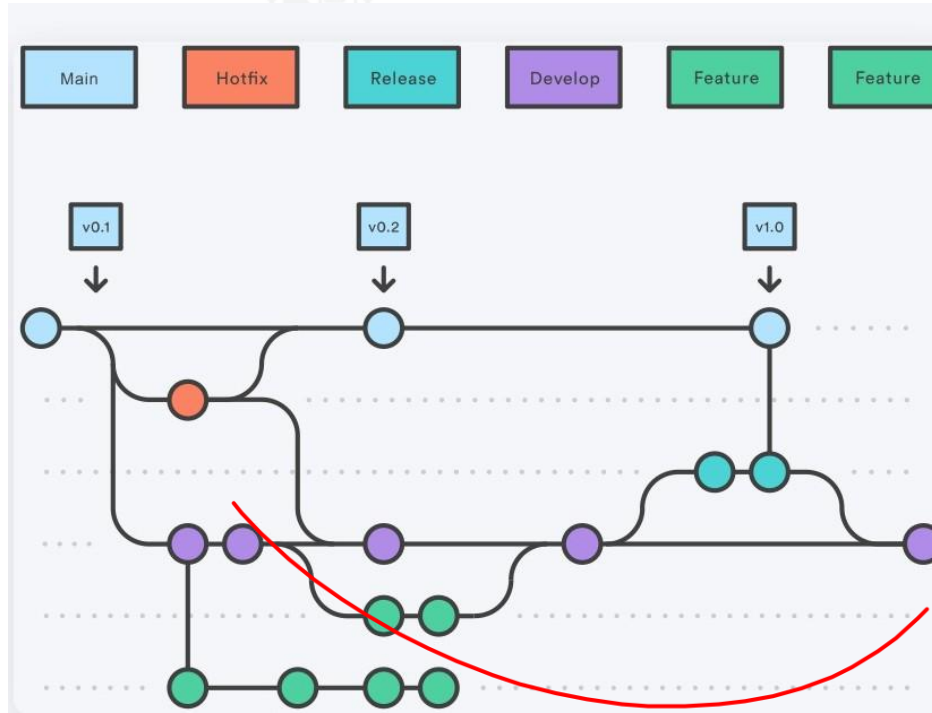
Crea instancias remotas del VCS alojadas externamente para conservar copias del código fuente

Protección

Almacenamiento

Disponibilidad

Introducción a Gitflow



¿Qué es Gitflow?

Flujo de trabajo de Git que define un **modelo estricto de ramificación** diseñado alrededor de la publicación del proyecto.



Propósito principal

Proporciona un marco sólido para gestionar proyectos más grandes con múltiples desarrolladores



Modelo de ramificación

Utiliza múltiples ramas especializadas para diferentes propósitos en el ciclo de desarrollo



Enfoque en publicación

Diseñado específicamente para facilitar el proceso de lanzamiento de versiones



Creador

Publicado y popularizado por Vincent Driessen en nvie.com

Ramas de desarrollo y maestras en Gitflow



Rama maestra **master**

Almacena el **historial de publicación oficial** del proyecto



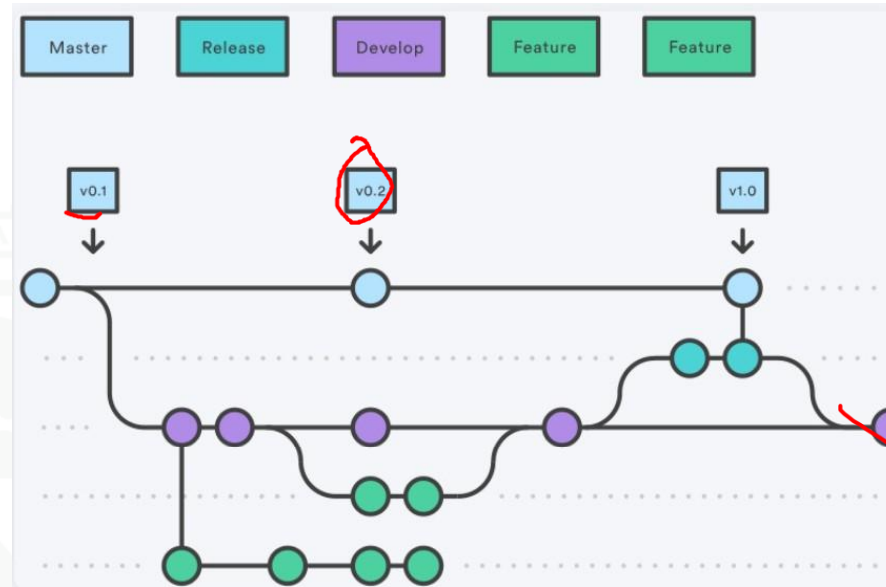
Rama de desarrollo **develop**

Sirve como **rama de integración** para todas las nuevas funcionalidades



Etiquetado de versiones

Todas las confirmaciones en la rama maestra deben tener un **número de versión**



Características clave



Historial de versiones estables



Flujo bidireccional entre ramas



Separación de código estable y en desarrollo

Ramas de función en Gitflow

1 Creación de rama

Cada nueva función se desarrolla en su propia rama bifurcada desde develop

2 Desarrollo

Los desarrolladores trabajan en la rama de función sin afectar el código principal

3 Fusión

Cuando la función está completa, se fusiona de nuevo en la rama develop

4 Regla fundamental

Las funciones nunca interactúan directamente con la rama maestra



💡 Ventajas clave

🛡️ Aislamiento del código

👥 Desarrollo paralelo

✅ Revisión antes de integración

Comandos básicos:

```
git checkout develop
git checkout -b feature/nueva-funcion
git checkout develop
git merge feature/nueva-funcion
```

Ramas de publicación y corrección en Gitflow



Ramas de publicación **release**

Se bifurcan de develop cuando hay suficientes funciones para una publicación

- ✓ Solo soluciones de errores
- ✓ Tareas orientadas a publicación
- ✓ Se fusiona en master y develop

Nombre



💡 Ventajas clave

- ✓ Estabilidad del código en producción
- ✓ Respuesta rápida a errores críticos
- ✓ Separación de desarrollo y correcciones

Comandos básicos:

```
git checkout master
git checkout -b hotfix/correccion-urgente
git checkout master
git merge hotfix/correccion-urgente
git tag v1.0.1
```


Ramas de publicación y corrección en Gitflow





Paradigmas de Diseño de Software

Introducción a los paradigmas de diseño

💡 ¿Qué son los paradigmas?

Enfoques fundamentales que guían la **estructura y organización** del software, estableciendo principios y patrones para resolver problemas de manera sistemática.



Base del diseño

Proporcionan un marco conceptual para organizar el código y estructurar sistemas complejos



Enfoque mental

Definen cómo los desarrolladores piensan y abordan la solución de problemas



Calidad y mantenibilidad

Mejoran la calidad, reutilización y mantenibilidad del software

PARADIGMA DE DISEÑO

DIFERENTES MANERAS DE PENSAR
PARA CREAR SOLUCIONES



Principales paradigmas



Estructurado



Orientado a objetos



Funcional



Lógico



Diseño estructurado - Definición y conceptos

¿Qué es el diseño estructurado?

Enfoque disciplinado para la **identificación, selección y organización** de módulos y sus relaciones, con el objetivo de reducir costos de mantenimiento.



Forma sigue función

La estructura del problema guía la forma de la solución



Descomposición modular

Segmentación del sistema en cajas negras organizadas jerárquicamente



Herramientas gráficas

Uso de diagramas de estructura para facilitar la comprensión



Estrategias derivadas

Métodos para derivar el diseño desde los resultados del análisis



Criterios de calidad

Estándares para evaluar y seleccionar la mejor solución

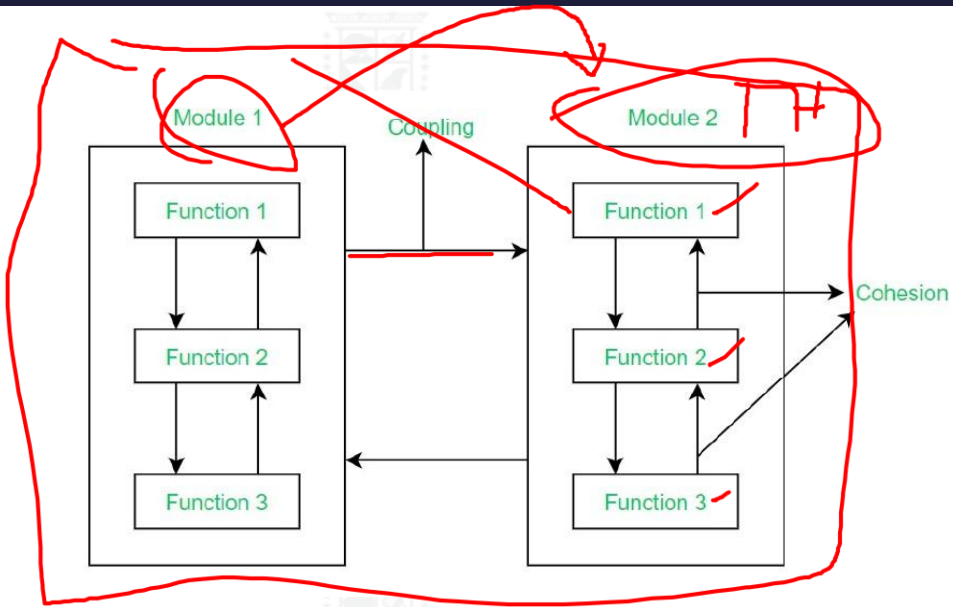


Orígenes



Presentado en la década de los '70 por Larry Constantine, continuado por Myers, Yourdon y Stevens

Diseño estructurado - Componentes



💡 Ventajas del diseño estructurado

- 👁 Claridad en la estructura
- 🔧 Mantenimiento simplificado
- 🧩 Reutilización de componentes



Módulos

Conjunto de instrucciones que ejecutan una **actividad específica**

- 📁 Entradas
- 📁 Salidas
- ⚙ Función
- ⌂ Lógica interna
- 📁 Estado interno



Invocaciones

Flechas que muestran las **llamadas entre módulos**, representando la estructura interna

- ➔ Direccionalidad
- ☰ Orden recomendado
- 👁 Interfaz expuesta



Cuplas

Datos comunicados entre módulos durante una **invocación**

- 📁 Datos
- ✏ Modificada
- ✓ Resultados

Diseño estructurado - Tipos de cuplas



Cupla de Datos Data Coupling

Transporta datos "**pueros**" entre módulos sin necesidad de conocer la lógica interna del receptor

- ✓ Valores independientes
- ✓ Sin dependencia lógica



Cupla Modificada Modified Coupling

Argumento enviado a un módulo que **modifica su valor**, representado con flecha doble

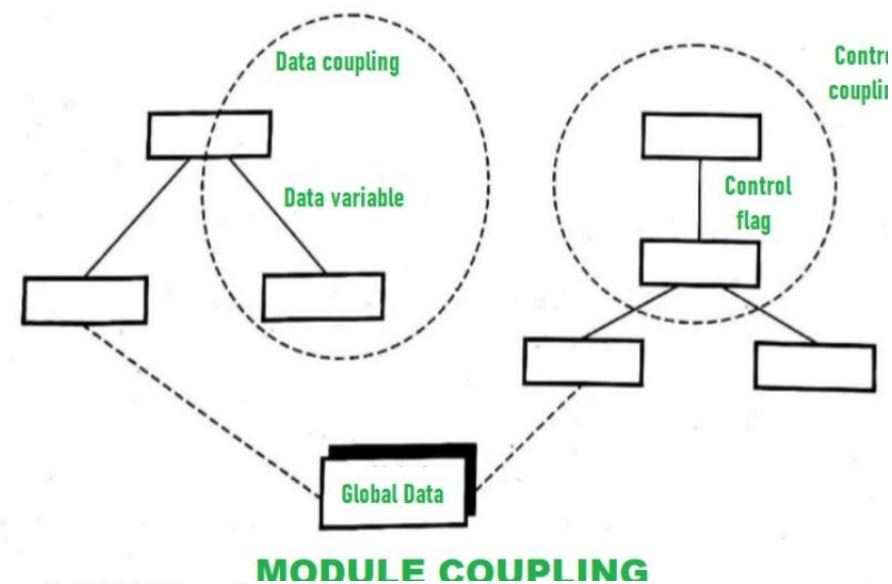
- ✓ Pasaje por referencia
- ✓ Valor modificado



Cupla de Resultados Result Coupling

Valores retornados **sin inicialización previa**, típicamente en funciones que generan resultados

- ✓ Valor de retorno
- ✓ Sin uso previo



Importancia de las cuplas

- 👁 Claridad en la comunicación
- 🔧 Facilita el mantenimiento
- 🧩 Permite la reutilización



Actividades

<https://online.utm.edu.ec/>



UNIVERSIDAD
TÉCNICA DE MANABÍ
Fundada en 1952

UTM
Unidos

